

Section 3: Introduction to Pytorch

What is Pytorch?

- PyTorch is an open source machine learning and deep learning framework.

What is Pytorch used for?

- PyTorch allows you to manipulate and process data and write machine learning algorithms using Python code.
- PyTorch also helps take care of many things such as GPU acceleration (making your code run faster) behind the scenes.

Other Options

- TensorFlow / Keras: Google's framework. Very similar to pytorch, helps researchers and developers build, train, and deploy ML models across a wide variety of platforms
- Scikit-Learn: Great for "traditional" ML (Random Forests, SVMs), but not designed for deep learning or GPU acceleration.

Imports

```
import numpy as np
import torch
import torch.nn as nn
```

Tensors

- Tensors are the fundamental building block of machine learning.
- Their job is to represent data in a numerical way.
- Think of a tensor as a multi-dimensional array, similar to a NumPy ndarray, but with the ability to run on GPUs.

```
# 1. Creating Tensors
x = torch.tensor([1.0, 2.0])
y = torch.ones(2, 2) # A 2x2 matrix of ones

print(x)
print("-----")
print(y)

tensor([1., 2.])
-----
tensor([[1., 1.],
        [1., 1.]])
```

```
# 2. Mathematical Operations
z = x + x
print(f"Addition: {z}")

Addition: tensor([2., 4.])

# 3. Matrix Multiplication
# 2 inputs and 3 hidden neurons
inputs = torch.tensor([0.5, -1.0])
weights = torch.randn(2, 3)
output = torch.matmul(inputs, weights)
print(f"Layer Output: {output}")

Layer Output: tensor([ 0.3708, -0.1033, -0.2769])
```

nn.Module

- The nn.Module is a base class in PyTorch for all neural network modules.
- holds all the weights, biases, and other sub-layers.
- It handles Forward and Backward logic.

Example 1:

1. Define the Architecture to match the board

```
import torch
import torch.nn as nn
import torch.optim as optim

class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        # Layer 1: Input (x1, x2) -> Hidden (h1, h2)
        # nn.Linear(2, 2) creates 4 weights (w1-w4) and 2 biases (b1)
        self.fc1 = nn.Linear(2, 2)

        # Layer 2: Hidden (h1, h2) -> Output (o1, o2)
        # nn.Linear(2, 2) creates 4 weights (w5-w8) and 2 biases (b2)
        self.fc2 = nn.Linear(2, 2)

        # Sigmoid Activation:  $\sigma(z) = 1 / (1 + e^{-z})$ 
        # This compresses values between 0 and 1
        self.sigmoid = nn.Sigmoid()

        # Manually initialize weights
        # torch.no_grad() is used because setting initial weights is
        # not a "calculus operation" we want to track.
        with torch.no_grad():
            # Hidden Layer: w1=0.15, w2=0.20, w3=0.25, w4=0.30 |
            # bias=0.35
            self.fc1.weight = nn.Parameter(torch.tensor([[0.15, 0.20],
```

```

[0.25, 0.30]]))
    self.fc1.bias = nn.Parameter(torch.tensor([0.35, 0.35]))

    # Output Layer: w5=0.40, w6=0.45, w7=0.50, w8=0.55 |
    bias=0.60
    self.fc2.weight = nn.Parameter(torch.tensor([[0.40, 0.45],
[0.50, 0.55]]))
    self.fc2.bias = nn.Parameter(torch.tensor([0.60, 0.60]))

    def forward(self, x):
        # Step 1: Calculate net input and activation for Hidden Layer
        # z_h = (x * weights) + bias -> out_h = sigmoid(z_h)
        x = self.sigmoid(self.fc1(x))

        # STEP 2: Calculate net input and activation for Output Layer
        # z_o = (out_h * weights) + bias -> out_o = sigmoid(z_o)
        x = self.sigmoid(self.fc2(x))

        return x # This presents our predictions o1, o2

```

1. Setup Data

```

# Input x1=0.05, x2=0.10
X = torch.FloatTensor([[0.05, 0.10]])

# Target T1=0.01, T2=0.99
y = torch.FloatTensor([[0.01, 0.99]])

```

1. Instantiate Model, Loss, and Optimizer

```

simple_model = SimpleNN()

# Loss Function: MSE measures the squared distance between Predicted
and Target
#  $E_{total} = \sum 1/2 (target - prediction)^2$ 
criterion = nn.MSELoss()

# Optimization Method: Stochastic Gradient Descent
# Updates a model's weights by calculating the error and "stepping" in
the direction that reduces that error
# It uses only one random data point (or a small "batch") at a time
rather than the entire dataset.
optimizer = optim.SGD(simple_model.parameters(), lr=0.5)

```

1. Training Loop (Single Step Verification)

```

print("--- Before Training ---")
initial_output = simple_model(X)
print(f"Initial Output: {initial_output.detach().numpy()}")
print(f"Initial Loss: {criterion(initial_output, y).item():.4f}")

```

```
--- Before Training ---  
Initial Output: [[0.75136507 0.7729285 ]]  
Initial Loss: 0.2984
```

Perform 1 Epoch (1 Forward and Backward Pass)

```
optimizer.zero_grad() # Clear the gradients  
outputs = simple_model(X) # Re-run the forward pass  
loss = criterion(outputs, y) # Calculate the loss  
loss.backward() # Backward: Pytorch calculates all partial derivatives  
automatically  
optimizer.step() # Update: The optimizer adjusts the weights using the  
gradients
```

```
print("\n--- After 1 Step of Gradient Descent ---")  
new_output = simple_model(X)  
print(f"New Output: {new_output.detach().numpy()}")  
print(f"New Loss: {criterion(new_output, y).item():.4f}")
```

```
--- After 1 Step of Gradient Descent ---  
New Output: [[0.7284418 0.7783769]]  
New Loss: 0.2805
```

Example 2: Iris Data

```
import torch  
import torch.nn as nn  
import torch.optim as optim  
from sklearn.datasets import load_iris  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler  
  
# Load the Iris dataset  
iris = load_iris()  
X, y = iris.data, iris.target  
  
print(X.shape)  
print(y.shape)  
  
(150, 4)  
(150,)
```

Splitting the data

```
# Split the dataset: 80% for training (learning), 20% for testing  
X_train, X_test, y_train, y_test = train_test_split(X, y,  
test_size=0.2, random_state=42)
```

Standardization

```
# This centers data at 0 with a standard deviation of 1.
# Neural networks learn much faster when all input features are on the
# same scale.
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Define the neural network architecture

```
class IrisNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(IrisNN, self).__init__()

        # Input Layer: Takes 4 features -> expands to 10 hidden
        # neurons
        # The number of hidden neurons is a hyperparameter that we
        # decide, meaning it's not calculated with a formula
        self.fc1 = nn.Linear(input_size, hidden_size)

        # Activation function: ReLU (Rectified Linear Unit): If x > 0,
        # return x; else return 0.
        # This is more popular than Sigmoid for deep networks because
        # it's faster.
        self.relu = nn.ReLU()

        # Output Layer: Takes 10 hidden neurons -> maps to 3 classes
        # (species)
        self.fc2 = nn.Linear(hidden_size, output_size)

        # The data "flows" through the network layers in order
    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

Set random seed for reproducibility

```
# Ensures we get the same random weights every time we run this
torch.manual_seed(42)

<torch._C.Generator at 0x7e1fe5543cd0>
```

Instantiate the neural network

```

# Parameters: 4 inputs, 10 hidden, 3 outputs (Iris Setosa, Versicolor, Virginica)
model = IrisNN(input_size=X.shape[1], hidden_size=10, output_size=3)

# Loss Function: standard for multi-class classification.
criterion = nn.CrossEntropyLoss()

# Optimization: Adam is a "smarter" version of SGD that adjusts the learning rate automatically.
optimizer = optim.Adam(model.parameters(), lr=0.01)

# Convert data to PyTorch tensors
X_train_tensor = torch.FloatTensor(X_train)
y_train_tensor = torch.LongTensor(y_train)

```

Train the Model

```

num_epochs = 100
for epoch in range(num_epochs):
    # Step 1: Forward Pass - Make a prediction
    outputs = model(X_train_tensor)

    # Step 2: Compute Loss
    loss = criterion(outputs, y_train_tensor)

    # Step 3: Zero Gradients
    optimizer.zero_grad()

    # Step 4: Backward Pass - Calculate the derivatives
    loss.backward()

    # Step 5: Optimizer Step - Update the weights based on the loss
    optimizer.step()

    # Print the loss every 10 epochs
    if (epoch+1) % 10 == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f}')

```

Epoch [10/100], Loss: 0.7783
Epoch [20/100], Loss: 0.5399
Epoch [30/100], Loss: 0.3921
Epoch [40/100], Loss: 0.2934
Epoch [50/100], Loss: 0.2166
Epoch [60/100], Loss: 0.1639
Epoch [70/100], Loss: 0.1284
Epoch [80/100], Loss: 0.1050
Epoch [90/100], Loss: 0.0902
Epoch [100/100], Loss: 0.0800

Evaluate the model

```
with torch.no_grad():
    X_test_tensor = torch.FloatTensor(X_test)
    y_test_tensor = torch.LongTensor(y_test)

    # Run the test data through the trained model
    outputs = model(X_test_tensor)

    # Get the index of the highest score (this is our predicted
    species)
    _, predicted = torch.max(outputs, 1)

    # Calculate accuracy: (Correct Predictions / Total)
    accuracy = (predicted == y_test_tensor).sum().item() /
len(y_test_tensor)
    print(f'Accuracy on the test set: {accuracy:.2f}')
```

Accuracy on the test set: 0.97